# Playwright Automation — Complete 30-Day Study Guide (With All Answers)

*All daily **Important questions** now include full answers below each question block.*

---

PLAYWRIGHT AUTOMATION Complete 30-Day Study Guide Beginner to job-oriented automation testing

30 days

2 hours/day

60 guided hours

JavaScript

Playwright Test

Practical projects

How every day works Hour 1: concept, plain-English explanation, syntax and demonstration. Hour 2: guided lab, solution pattern, self-check and homework. Includes a 30-day schedule, daily examples and answer patterns, four guided project tracks, a final LMS/admin-dashboard capstone, interview questions, revision sheets, troubleshooting and a command reference.

Important: URLs, labels, API paths and response shapes in examples are illustrative. Adjust them to your own permitted test application. Never run destructive tests against production or commit passwords, tokens, real customer data or authentication state.

---

How to use this guide Type the examples instead of only reading them. Run the test, deliberately introduce one mistake, inspect the failure, fix it, and write down what the assertion proves. The goal is to build debugging ability and explain your choices.

Recommended daily two-hour rhythm Time

Activity

Evidence

00:00–00:10

Recall yesterday; define today's goal.

Explain the goal in one sentence.

00:10–00:40

Learn the concept and type the demo.

Predict what the code should do.

00:40–01:00

Review failure modes and debug.

Explain one common mistake.

01:00–01:15

Complete guided lab.

Make a small test pass.

01:15–01:50

Independent challenge.

Write code without copying line-by-line.

01:50–02:00

Review, commit, note blockers.

Record output and next step.

30-day roadmap Week 1: Fundamentals: setup, JavaScript, browser model, locators, actions, assertions and login Week 2: Practical web automation: runner, waits, controls, dialogs, files, debugging and form mini-project Week 3: Reusable framework: POM, fixtures, data-driven tests, authentication, API, mocking and isolation Week 4: Industry workflow: risk-based testing, e-commerce, CRUD, cross-browser, responsive, CI and capstone

---

Setup checklist Install a supported Node.js LTS version, VS Code, Git and a browser. Start from the Playwright initializer so the project includes configuration, dependencies and sample tests. node -v npm -v npm init playwright@latest

# Select JavaScript

npx playwright install npx playwright --version npx playwright test npx playwright show-report

Typical project structure project/ tests/ example.spec.js pages/ fixtures/ test-data/ playwright.config.js package.json package-lock.json .gitignore README.md

Use `npm init playwright@latest` for the current initializer and follow its prompts. Verify the official Playwright documentation for current Node.js and browser requirements because these can change.

---

Day 01 — Setup, installation and your first test Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Playwright is a browser automation framework. Playwright Test adds a test runner, assertions, fixtures, parallel execution and reports.

Example to understand const { test, expect } = require('@playwright/test'); test('homepage has a title', async ({ page }) => { await page.goto('https://example.com'); await expect(page).toHaveTitle(/Example Domain/); await expect(page.getByRole('heading', { name: 'Example Domain' })).toBeVisible(); });

Hour 2 — Hands-on task Install Node.js LTS and VS Code. Initialize a JavaScript Playwright project, run the generated test, then create one test for example.com.

Answer / solution pattern

# Terminal

node -v npm -v npm init playwright@latest

# Choose JavaScript when prompted

npx playwright install npx playwright test npx playwright show-report

Important questions

1. What is Playwright Test?
2. What does the test fixture `page` provide?
3. How do you run one spec file?

**Answers**

1. **What is Playwright Test?**

   Playwright Test is Microsoft's test runner for Playwright: it provides `test` / `expect`, fixtures (`page`, `request`), hooks, parallel workers, retries, reporters, and HTML/trace output.

2. **What does the test fixture `page` provide?**

   The `page` fixture is a fresh browser tab (Page) in an isolated BrowserContext for that test. Playwright creates it before the test and closes it after, so you get a clean session without manual browser launch.

3. **How do you run one spec file?**

   Run one file with `npx playwright test path/to/file.spec.js` (or `.ts`). You can also use `-g "test title"` to run a single test inside the file.

Revision notes • Use a supported Node.js LTS version. • The test file usually ends in `.spec.js`. • A test passes only when its actions and assertions complete without failure.

---

Day 02 — JavaScript essentials for automation Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Automation code uses variables, arrays, objects, functions, loops, conditions, promises and async/await. Browser operations are asynchronous, so await them.

Example to understand const users = [ { username: 'amina', active: true }, { username: 'pavan', active: false }, { username: 'maaz', active: true } ]; for (const user of users) { if (user.active) console.log(user.username); }

Hour 2 — Hands-on task Create three test users as objects. Print active users, then loop through the array and check each user using a test.

Answer / solution pattern const { test, expect } = require('@playwright/test'); const users = [ { name: 'Amina', expected: 'Welcome, Amina' }, { name: 'Pavan', expected: 'Welcome, Pavan' } ]; for (const user of users) { test( `welcome message for ${user.name}` , async ({ page }) => { await page.goto('https://example.com'); await expect(page.getByRole('heading')).toBeVisible(); expect(user.expected).toContain(user.name); }); }

Important questions

1. Why do Playwright actions usually use `await`?
2. What is the difference between an array and an object?
3. When is a `for...of` loop useful?

**Answers**

1. **Why do Playwright actions usually use `await`?**

   Browser operations return Promises; `await` waits for the action to finish so the next line does not run on a half-loaded page or mid-click state.

2. **What is the difference between an array and an object?**

   An array is an ordered list of values (e.g. several users). An object is one record with named fields (e.g. one user's `username` and `active` flag).

3. **When is a `for...of` loop useful?**

   Use `for...of` when you need to loop a collection and `await` inside the loop—e.g. data-driven tests—without callback/async pitfalls of `forEach`.

Revision notes • Use `const` by default and `let` when reassignment is needed. • Objects represent one record; arrays hold collections. • Do not use `forEach(async ...)` when you need to reliably await each operation.

---

Day 03 — Browser, context, page and test lifecycle Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Browser is the browser engine instance; BrowserContext is an isolated session with its own cookies/storage; Page is a tab. The `page` fixture is created for a test and cleaned up by the runner.

Example to understand const { test, expect } = require('@playwright/test'); test('two isolated sessions', async ({ browser }) => { const contextA = await browser.newContext(); const pageA = await contextA.newPage(); await pageA.goto('https://example.com'); const contextB = await browser.newContext(); const pageB = await contextB.newPage(); await pageB.goto('https://example.com'); await expect(pageA).toHaveURL(/example.com/); await contextA.close(); await contextB.close(); });

Hour 2 — Hands-on task Open two pages in separate contexts. Explain which state is shared and which is isolated.

Answer / solution pattern const { test, expect } = require('@playwright/test'); test('page navigation', async ({ page }) => { await page.goto('https://example.com'); await expect(page).toHaveURL('https://example.com/'); });

Important questions

1. Compare Browser, BrowserContext and Page.
2. Why isolate tests?
3. What does the `page` fixture manage for you?

**Answers**

1. **Compare Browser, BrowserContext and Page.**

   Browser = engine instance; BrowserContext = isolated session (cookies, storage, permissions); Page = one tab/window inside a context. Multiple contexts do not share cookies unless you copy state.

2. **Why isolate tests?**

Isolation prevents tests from polluting each other's login, cart, or DB rows, so parallel runs and reruns stay deterministic.

3. **What does the `page` fixture manage for you?**

   It launches (or reuses) browser/context, gives you a new Page, and tears down after the test so you do not leak memory or session state.

Revision notes • Contexts isolate cookies and local storage. • Prefer built-in fixtures unless custom lifecycle control is required. • A tab is a Page, not a BrowserContext.

---

Day 04 — Locators and robust element selection Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Locators describe how to find elements and are re-evaluated when used. Prefer user-facing locators such as role, label and placeholder; use test IDs when appropriate. Avoid brittle CSS/XPath chains.

Example to understand const heading = page.getByRole('heading', { name: 'Sign in' }); const email = page.getByLabel('Email address'); const submit = page.getByRole('button', { name: 'Sign in' }); await expect(heading).toBeVisible(); await email.fill('learner@example.com'); await submit.click();

Hour 2 — Hands-on task On a practice form, locate heading, email, password and submit button. Use accessible locators and make the name exact when necessary.

Answer / solution pattern // Example markup should expose labels: Email address Sign in // Test: await page.getByLabel('Email address').fill('learner@example.com'); await page.getByRole('button', { name: 'Sign in' }).click();

Important questions

1. Why is `getByRole()` often preferred?
2. What is a strict-mode violation?
3. When is `getByTestId()` useful?

**Answers**

1. **Why is `getByRole()` often preferred?**

   It matches how users and assistive tech see controls (button, link, textbox) and is usually more stable than CSS classes or DOM depth.

2. **What is a strict-mode violation?**

   Your locator matches more than one element when Playwright expects a single target for an action. Narrow with role+name, container, or `filter()`.

3. **When is `getByTestId()` useful?**

   When the team adds `data-testid` (or similar) as a stable contract between dev and QA, especially for custom widgets without good labels.

Revision notes • Ambiguous locators should be made specific, not blindly forced. • Accessible names improve testability and usability. • Avoid selectors tied to generated class names.

---

Day 05 — Actions and form automation Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Use `fill()` for text fields, `check()` / `uncheck()` for checkboxes, `selectOption()` for native selects, and `click()` for buttons. Prefer the action matching the control.

Example to understand await page.getByLabel('Full name').fill('Amina Khan'); await page.getByLabel('Email').fill('amina@example.com'); await page.getByLabel('Password').fill('SafePass123!'); await page.getByLabel('Accept terms').check(); await page.getByRole('button', { name: 'Create account' }).click();

Hour 2 — Hands-on task Automate a registration form with name, email, password, terms checkbox and submit. Assert the success message.

Answer / solution pattern const { test, expect } = require('@playwright/test'); test('registration form shows confirmation', async ({ page }) => { await page.goto('http://localhost:3000/register'); await page.getByLabel('Full name').fill('Amina Khan'); await page.getByLabel('Email').fill('amina@example.com'); await page.getByLabel('Password').fill('SafePass123!'); await page.getByLabel('Accept terms').check(); await page.getByRole('button', { name: 'Create account' }).click(); await expect(page.getByRole('status')).toContainText('Account created'); });

Important questions

1. When should you use `fill()` instead of `type()` ?
2. How do you select an option from a native select?
3. How do you check a checkbox?

**Answers**

1. **When should you use `fill()` instead of `type()` ?**

   `fill()` sets the input value in one step (clear + type)—best for most fields. `type()` / `pressSequentially()` simulates keystrokes when the app validates per key.

2. **How do you select an option from a native select?**

   On a native `<select>` : `await page.getByLabel('Course').selectOption({ label: 'Playwright' })` or by `value` / index.

3. **How do you check a checkbox?**

   `await page.getByLabel('Accept terms').check()` then `await expect(locator).toBeChecked()` to verify state.

Revision notes • Match the locator to a real label in the application. • Use test-only data and an approved test environment. • An action succeeding does not prove the application behaved correctly; assert the outcome.

---

Day 06 — Assertions and expected results Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Assertions make a test meaningful. Web-first assertions such as `toBeVisible()` retry until success or timeout; ordinary assertions like `expect(value).toBe(...)` evaluate immediately.

Example to understand await expect(page.getByRole('heading', { name: 'Dashboard' })).toBeVisible(); await expect(page).toHaveURL(/dashboard/); await expect(page.getByTestId('balance')).toHaveText('$120.00'); expect(2 + 2).toBe(4);

Hour 2 — Hands-on task Add at least two assertions to each of three tests: visible result, URL or field value. Include one negative assertion.

Answer / solution pattern await expect(page.getByText('Saved successfully')).toBeVisible(); await expect(page.getByRole('button', { name: 'Save' })).toBeEnabled(); await expect(page.getByText('Invalid credentials')).toHaveCount(0);

Important questions

1. What makes an assertion web-first?
2. Why await `expect(locator).toBeVisible()`?
3. What is the difference between checking an action and checking its result?

**Answers**

1. **What makes an assertion web-first?**

   Playwright's locator assertions (e.g. `toBeVisible()`) retry until the condition is true or the timeout expires—they wait for the UI, not a single instant snapshot.

2. **Why await `expect(locator).toBeVisible()`?**

   The matcher is async and polls the DOM; `await` ensures you wait for the assertion to finish and fail with a useful message if the element never appears.

3. **What is the difference between checking an action and checking its result?**

   Checking an action only proves the click ran; checking the result proves the app showed the right message, URL, or data—the test's real goal.

Revision notes • Assert user-visible behavior, not implementation details. • Avoid checking only that a click completed. • Use negative assertions for important absence conditions.

---

Day 07 — Test cases, suites and login automation Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words A test case has preconditions, steps, test data and expected results. A suite groups related tests. Login testing should include valid credentials, invalid credentials and required-field validation.

Example to understand const { test, expect } = require('@playwright/test'); test.describe('login', () => { test('shows an error for invalid credentials', async ({ page }) => { await page.goto('http://localhost:3000/login'); await page.getByLabel('Email').fill('wrong@example.com'); await page.getByLabel('Password').fill('WrongPass123'); await page.getByRole('button', { name: 'Sign in' }).click(); await expect(page.getByRole('alert')).toContainText('Invalid email or password'); }); });

Hour 2 — Hands-on task Write a test-case table for login: valid login, invalid password, blank fields and logout. Implement two cases.

Answer / solution pattern test('blank login shows validation', async ({ page }) => { await page.goto('http://localhost:3000/login'); await page.getByRole('button', { name: 'Sign in' }).click(); await expect(page.getByText('Email is required')).toBeVisible(); });

Important questions

1. What belongs in a good test case?
2. Name four login scenarios.
3. Why should tests not depend on execution order?

**Answers**

1. **What belongs in a good test case?**

   Clear title, preconditions, steps, test data, expected observable outcome, and cleanup/isolation notes.

2. **Name four login scenarios.**

   Valid login, invalid password, blank/required fields, locked/disabled account (if applicable)—plus logout/session cleared.

3. **Why should tests not depend on execution order?**

   Ordered suites hide dependencies; any test should pass alone so parallel CI and debugging stay trustworthy.

Revision notes • Cover positive, negative and boundary cases. • Do not store real passwords in source control. • Keep tests independent and repeatable.

Weekly checkpoint — Week 1 Explain browser/context/page, write locators and actions, and create a login suite with meaningful assertions.

---

Day 08 — Configuration, runner, hooks and baseURL Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words `playwright.config.js` centralizes test directory, base URL, timeouts, reporters and projects. Hooks prepare/clean state. `beforeEach` runs before each test; use `beforeAll` only for genuinely shared setup.

Example to understand const { defineConfig } = require('@playwright/test'); module.exports = defineConfig({ testDir: './tests', timeout: 30_000, use: { baseURL: 'http://localhost:3000', headless: true }, reporter: [['list'], ['html', { open: 'never' }]] });

Hour 2 — Hands-on task Move the app URL into `baseURL` ; update tests to use relative paths. Add a `beforeEach` navigation hook.

Answer / solution pattern const { test, expect } = require('@playwright/test'); test.describe('home page', () => { test.beforeEach(async ({ page }) => { await page.goto('/'); }); test('shows main heading', async ({ page }) => { await expect(page.getByRole('heading', { level: 1 })).toBeVisible(); }); });

Important questions

1. What belongs in the config file?
2. Compare `beforeEach` and `beforeAll` .
3. Why use `baseURL` ?

**Answers**

1. **What belongs in the config file?**

testDir, timeouts, baseURL, reporters, browser projects, retries, trace/screenshot/video, `use` defaults, and optional globalSetup.

2. **Compare `beforeEach` and `beforeAll`.**

   `beforeEach` runs before every test (fresh navigation/login). `beforeAll` runs once per worker—risky if tests mutate shared state.

3. **Why use `baseURL`?**

   Relative URLs ( `page.goto('/login')` ) work across local/staging by changing one config value instead of every test file.

Revision notes • Use hooks for repeatable setup, not to hide test behavior. • A test should remain understandable in isolation. • Run `npx playwright show-report` to inspect the HTML report.

---

Day 09 — Auto-waiting, timeouts and dynamic elements Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Playwright waits for actionability before actions and retries web-first assertions. A click generally needs a unique element that is visible, stable, enabled and able to receive events. Fixed sleeps make tests slower and flaky.

Example to understand await page.getByRole('button', { name: 'Load results' }).click(); await expect(page.getByRole('table')).toBeVisible(); await expect(page.getByRole('row', { name: /Amina Khan/ })).toBeVisible();

Hour 2 — Hands-on task Test a button that loads content asynchronously. Replace any `waitForTimeout()` with a meaningful assertion.

Answer / solution pattern // Preferred synchronization await page.getByRole('button', { name: 'Load results' }).click(); await expect(page.getByText('Results loaded')).toBeVisible(); // Avoid as a routine solution: // await page.waitForTimeout(3000);

Important questions

1. What is actionability?
2. Why are fixed sleeps fragile?
3. How do you handle a genuinely slow response?

**Answers**

1. **What is actionability?**

   An element is ready for action: attached, visible, stable, enabled, and receiving events (not covered by another element).

2. **Why are fixed sleeps fragile?**

   They wait a fixed time regardless of app speed—too short flakes, too long wastes CI minutes and masks missing waits.

3. **How do you handle a genuinely slow response?**

   Use a web-first assertion ( `expect(...).toBeVisible()` ), `waitForURL`, or `waitForResponse`; increase timeout only after confirming the app is legitimately slow.

Revision notes • Wait for the state you need, not an arbitrary duration. • Increase timeout only when justified by the system. • A timeout is a symptom; inspect the trace and locator.

---

Day 10 — Dropdowns, checkboxes, radios, tables and lists Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Native selects support `selectOption`. Checkboxes use `check` and `uncheck`. Radio buttons use `check`. Tables should be located by semantic rows and cells, not brittle positional selectors.

Example to understand await page.getByLabel('Course').selectOption({ label: 'Playwright' }); await page.getByLabel('Receive updates').check(); await page.getByLabel('Beginner').check(); await expect(page.getByRole('row', { name: /Amina.*Playwright/ })).toBeVisible();

Hour 2 — Hands-on task Automate a course enrollment form containing a select, checkbox and radio button; then verify a matching row.

Answer / solution pattern await page.getByLabel('Course').selectOption('playwright'); await page.getByLabel('Receive updates').check(); await page.getByLabel('Beginner').check(); await page.getByRole('button', { name: 'Enroll' }).click(); await expect(page.getByRole('row', { name: /Amina Khan.*Playwright/ })).toBeVisible();

Important questions

1. How do you select an option by label?
2. How do you target a row by its content?
3. Why are semantic table locators useful?

**Answers**

1. **How do you select an option by label?**

   `selectOption({ label: 'Playwright' })` or `selectOption('value')` on the select locator.

2. **How do you target a row by its content?**

   `page.getByRole('row', { name: /Amina.*Playwright/ })` or `getByRole('row').filter({ hasText: '...' })`.

3. **Why are semantic table locators useful?**

   Rows/cells tied to user-visible text survive layout changes better than `tr:nth-child(3) td:nth-child(2)`.

Revision notes • Use `selectOption()` for native `<select>` controls. • Custom dropdowns may require clicking options instead. • Prefer a row locator based on meaningful content.

---

Day 11 — Dialogs, popups, tabs and multiple pages Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Register dialog handlers before the triggering action. Use `page.expectPopup()` to capture a new tab. Use `context.pages()` only when you need to inspect multiple open pages.

Example to understand page.once('dialog', async dialog => { expect(dialog.type()).toBe('confirm'); await dialog.accept(); }); await page.getByRole('button', { name: 'Delete record' }).click();

Hour 2 — Hands-on task Write tests for accepting and dismissing a confirmation dialog. Add a link that opens a popup and assert its URL.

Answer / solution pattern const popupPromise = page.waitForEvent('popup'); await page.getByRole('link', { name: 'Open help' }).click(); const popup = await popupPromise; await expect(popup).toHaveURL(/help/);

Important questions

1. Why must a dialog listener be registered before clicking?
2. How do you capture a popup?
3. What is the difference between a tab and a context?

**Answers**

1. **Why must a dialog listener be registered before clicking?**

   The dialog event fires synchronously when the dialog appears; if no handler is registered yet, the test can hang or auto-dismiss unpredictably.

2. **How do you capture a popup?**

   `const [popup] = await Promise.all([page.waitForEvent('popup'), clickTrigger])` (or `context.waitForEvent('page')`).

3. **What is the difference between a tab and a context?**

   A tab is a Page in the same BrowserContext (shared cookies). A new context is a separate session—like another user or incognito.

Revision notes • Use `once` for a one-time dialog handler. • Wait for the popup event before the triggering action. • Assert what the user sees in the new page.

---

Day 12 — Uploads, downloads and file handling Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Use `setInputFiles()` for uploads. For downloads, register `waitForEvent('download')` before clicking and inspect the suggested filename. Use small test fixtures and avoid sensitive files.

Example to understand await page.getByLabel('Profile photo').setInputFiles('tests/fixtures/avatar.png'); const downloadPromise = page.waitForEvent('download'); await page.getByRole('button', { name: 'Export CSV' }).click(); const download = await downloadPromise; expect(download.suggestedFilename()).toMatch(/.csv$/);

Hour 2 — Hands-on task Upload a fixture file and verify success. Trigger a download and assert the filename extension.

Answer / solution pattern const path = require('path'); await page.getByLabel('Attachment').setInputFiles( path.join(__dirname, 'fixtures', 'sample.txt') ); await expect(page.getByText('sample.txt')).toBeVisible();

Important questions

1. How do you upload a file?
2. Why start waiting for download before clicking?
3. Where should test fixtures live?

**Answers**

1. **How do you upload a file?**

   `await locator.setInputFiles('path/to/file')` on the file input (hidden inputs are OK).

2. **Why start waiting for download before clicking?**

   The download starts after the click; registering the waiter first avoids missing a fast download event.

3. **Where should test fixtures live?**

   In `tests/fixtures/` (or `test-data/`)—small, non-secret files committed to the repo; never real customer documents.

Revision notes • Use repository-controlled fixture files. • Do not commit confidential files or downloaded secrets. • Handle download failures and cleanup where necessary.

---

Day 13 — Debugging, traces and HTML reports Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Use headed mode, UI mode, Inspector, screenshots, traces and HTML reports to understand failures. A trace can show actions, snapshots, network activity and timing.

Example to understand // playwright.config.js module.exports = { use: { trace: 'retain-on-failure', screenshot: 'only-on-failure', video: 'retain-on-failure' } };

Hour 2 — Hands-on task Intentionally break a locator, run the test, inspect the error and trace, fix it, and record the root cause.

Answer / solution pattern npx playwright test --headed npx playwright test --debug npx playwright test --trace on npx playwright show-report npx playwright show-trace trace.zip

Important questions

1. What can a trace show?
2. How do you debug a test interactively?
3. Why retain traces on failure?

**Answers**

1. **What can a trace show?**

   Action log, DOM snapshots, network, console, timing, and screenshots/videos linked to each step.

2. **How do you debug a test interactively?**

   `npx playwright test --debug` or `--ui`, or set `PWDEBUG=1`—step through with Inspector.

3. **Why retain traces on failure?**

   Failures in CI are hard to reproduce locally; traces are the fastest way to see locators, network, and overlays at the failing moment.

Revision notes • Reproduce before changing multiple things. • Read the first meaningful error and inspect the failed action. • Do not commit trace files containing sensitive data.

---

Day 14 — Mini-project: robust form suite Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Combine locators, actions, assertions, validation and reporting into a small maintainable suite. Cover success, invalid input and required fields.

Example to understand test('invalid email is rejected', async ({ page }) => { await page.goto('/register'); await page.getByLabel('Email').fill('not-an-email'); await page.getByRole('button', { name: 'Create account' }).click(); await expect(page.getByText('Enter a valid email')).toBeVisible(); });

Hour 2 — Hands-on task Deliver a 5-test registration/form suite with clear names, reusable baseURL, meaningful assertions and an HTML report.

Answer / solution pattern test('terms are required', async ({ page }) => { await page.goto('/register'); await page.getByLabel('Full name').fill('Amina Khan'); await page.getByLabel('Email').fill('amina@example.com'); await page.getByLabel('Password').fill('SafePass123!'); await page.getByRole('button', { name: 'Create account' }).click(); await expect(page.getByText('Accept the terms')).toBeVisible(); });

Important questions

1. Which scenarios should the form suite cover?
2. What makes a test maintainable?
3. What evidence should accompany the project?

**Answers**

1. **Which scenarios should the form suite cover?**

   Happy path, invalid input, required fields, boundary values, and at least one negative assertion (error message or count 0).

2. **What makes a test maintainable?**

   Stable locators, clear names, no shared mutable data, config baseURL, and assertions on outcomes —not only actions.

3. **What evidence should accompany the project?**

   HTML report screenshot, list of tests run, README with setup/commands, and notes on environment assumptions.

Revision notes • Week 2 checkpoint: a form suite that can be rerun reliably. • Include README, test commands and known assumptions. • Keep test data clearly separated from test logic.

Weekly checkpoint — Week 2 Run a form suite, replace fixed sleeps with state-based waits, and inspect a report or trace.

---

Day 15 — Page Object Model (POM) Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words POM puts page-specific locators and actions in a class. Tests describe behavior; page objects provide reusable operations. Avoid making page objects so generic that they hide what the test does.

Example to understand class LoginPage { constructor(page) { this.page = page; this.email = page.getByLabel('Email'); this.password = page.getByLabel('Password'); this.submit = page.getByRole('button', { name: 'Sign in' }); } async open() { await this.page.goto('/login'); } async login(email, password) { await this.email.fill(email); await this.password.fill(password); await this.submit.click(); } } module.exports = { LoginPage };

Hour 2 — Hands-on task Create `pages/login.page.js` and a test that uses it. Keep assertions in the test unless they are page-specific reusable checks.

Answer / solution pattern const { test, expect } = require('@playwright/test'); const { LoginPage } = require('../pages/login.page'); test('invalid login shows error', async ({ page }) => { const login = new LoginPage(page); await login.open(); await login.login('wrong@example.com', 'bad-password'); await expect(page.getByRole('alert')).toContainText('Invalid'); });

Important questions

1. What belongs in a page object?
2. What is a downside of overusing POM?
3. Where should the main behavior assertion live?

**Answers**

1. **What belongs in a page object?**

   Locators for that page and user-like methods ( `login` , `open` , `search` )—not the whole test scenario or unrelated pages.

2. **What is a downside of overusing POM?**

   Tests become hard to read; you jump through layers to see what is being verified. Keep assertions in specs unless truly reusable.

3. **Where should the main behavior assertion live?**

   In the test file: the test should state the expected user-visible outcome ( `expect` on alert, URL, row, etc.).

Revision notes • One page object per meaningful page/component is a useful starting point. • Expose actions users understand: `login` , `search` , `save` . • Keep tests readable and assertions meaningful.

---

Day 16 — Fixtures and reusable setup Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Fixtures provide dependencies to tests and manage their lifecycle. Built-in fixtures include `page` , `context` , `browser` and `request` . Custom fixtures can construct page objects or prepare test-specific resources.

Example to understand const base = require('@playwright/test'); const { LoginPage } = require('../pages/login.page'); exports.test = base.test.extend({ loginPage: async ({ page }, use) => { await use(new LoginPage(page)); } }); exports.expect = base.expect;

Hour 2 — Hands-on task Create a custom `loginPage` fixture and use it in two tests. Avoid global mutable state shared by parallel tests.

Answer / solution pattern const { test, expect } = require('../fixtures/test'); test('login page opens', async ({ loginPage, page }) => { await loginPage.open(); await expect(page.getByRole('heading', { name: 'Sign in' })).toBeVisible(); });

Important questions

1. What problem do fixtures solve?
2. Why is global mutable test data dangerous?

3. When should setup be placed in a fixture?

**Answers**

1. **What problem do fixtures solve?**

   Dependency injection with lifecycle—setup before `use()`, teardown after—so tests declare what they need without copy-paste.

2. **Why is global mutable test data dangerous?**

   Parallel workers overwrite the same account name or counter, causing random pass/fail.

3. **When should setup be placed in a fixture?**

   When multiple tests need the same constructed object (page object, authenticated context) with consistent setup/teardown.

Revision notes • Fixtures clarify dependencies and cleanup. • Avoid hidden setup that makes tests hard to understand. • Each test should receive isolated resources where practical.

---

Day 17 — Data-driven testing and boundary values Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Data-driven testing runs one behavior with several data sets. Cover valid values, invalid values, empty input, minimum/maximum boundaries and special characters without duplicating entire tests.

Example to understand const cases = [ { email: 'valid@example.com', valid: true }, { email: 'missing-at.example.com', valid: false }, { email: '', valid: false } ]; for (const item of cases) { test( `email validation: ${item.email || 'empty'}` , async ({ page }) => { await page.goto('/register'); await page.getByLabel('Email').fill(item.email); await page.getByRole('button', { name: 'Create account' }).click(); if (item.valid) await expect(page.getByText('Enter a valid email')).toHaveCount(0); else await expect(page.getByText('Enter a valid email')).toBeVisible(); }); }

Hour 2 — Hands-on task Build a table of at least six password cases and automate validation for short, long, blank and valid passwords.

Answer / solution pattern const passwords = [ { value: '', valid: false }, { value: 'abc', valid: false }, { value: 'StrongPass123!', valid: true } ]; for (const item of passwords) { test( `password case ${item.value || 'blank'}` , async ({ page }) => { await page.goto('/register'); await page.getByLabel('Password').fill(item.value); await page.getByRole('button', { name: 'Create account' }).click(); if (item.valid) { await expect(page.getByText('Password is too short')).toHaveCount(0); } else { await expect(page.getByText('Password is too short')).toBeVisible(); } }); }

Important questions

1. What is a boundary-value test?
2. Why use data-driven tests?
3. What makes a test title useful?

**Answers**

1. **What is a boundary-value test?**

Testing at edges of allowed input—min/max length, empty, just below/above limits—where bugs often appear.

2. **Why use data-driven tests?**

   One test logic, many data rows—less duplication and clearer failure names per case.

3. **What makes a test title useful?**

   Includes the case identifier and intent: `email validation: missing-at` so the report shows which row failed.

Revision notes • Do not assume all validation messages are identical across apps. • Keep cases small and named so failures are easy to identify. • Cover boundaries, not only typical inputs.

---

Day 18 — Authentication and storage state Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Storage state can preserve cookies and local storage so tests can begin authenticated. Treat it as a credential: keep it out of source control, rotate it and use a dedicated test account.

Example to understand // Setup once with a dedicated test account, then save state. await page.goto('/login'); await page.getByLabel('Email').fill(process.env.E2E_USER); await page.getByLabel('Password').fill(process.env.E2E_PASSWORD); await page.getByRole('button', { name: 'Sign in' }).click(); await page.context().storageState({ path: 'playwright/.auth/user.json' });

Hour 2 — Hands-on task Configure a local authenticated state using a dedicated test account. Add the auth folder to `.gitignore` and assert an authenticated page loads.

Answer / solution pattern // playwright.config.js use: { storageState: 'playwright/.auth/user.json' } // .gitignore playwright/.auth/ .env

Important questions

1. What can storage state contain?
2. Why should auth state not be committed?
3. When should tests log in through the UI instead?

**Answers**

1. **What can storage state contain?**

   Typically cookies and localStorage (and similar) serialized to JSON so a new context can skip UI login.

2. **Why should auth state not be committed?**

   It is equivalent to live session credentials—anyone with the file can act as that user; rotate if leaked.

3. **When should tests log in through the UI instead?**

   For the login flow itself, MFA changes, or when you must prove the sign-in UI end-to-end.

Revision notes • Use a separate account with minimal permissions. • Never paste tokens into logs or public repositories. • Regenerate state when it becomes invalid.

---

Day 19 — API testing with Playwright request Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words API tests check HTTP behavior directly. Use `request.get/post/put/delete`, inspect status and response JSON, and validate important fields. API tests are often faster than UI tests.

Example to understand const { test, expect } = require('@playwright/test'); test('GET students returns a list', async ({ request }) => { const response = await request.get('/api/students'); expect(response.ok()).toBeTruthy(); expect(response.status()).toBe(200); const body = await response.json(); expect(Array.isArray(body.data)).toBeTruthy(); });

Hour 2 — Hands-on task Write a GET test and a POST test for a local test API. Adapt endpoint and response shape to your actual application.

Answer / solution pattern test('create a student', async ({ request }) => { const response = await request.post('/api/students', { data: { name: 'Playwright Learner', email: `pw-${Date.now()}@example.test` } }); expect(response.status()).toBe(201); const body = await response.json(); expect(body.data.name).toBe('Playwright Learner'); });

Important questions

1. How do you assert an API response status?
2. Why check the response body as well?
3. What is a safe way to generate unique test data?

**Answers**

1. **How do you assert an API response status?**

   `expect(response.status()).toBe(200)` or `expect(response.ok()).toBeTruthy()` for success range.

2. **Why check the response body as well?**

   Status alone can be 200 with wrong JSON; assert schema fields the UI or contract depends on.

3. **What is a safe way to generate unique test data?**

   `Date.now()`, UUID, or `pw-${Date.now()}@example.test` emails so parallel runs do not collide.

Revision notes • Endpoint paths and JSON shapes are application-specific. • Use test environments and clean up created records. • Do not test destructive endpoints against production.

---

Day 20 — Mocking and network interception Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words `page.route()` can intercept requests and fulfill them with mock responses, modify requests or abort traffic. Mocking makes UI tests deterministic but does not prove the real backend works.

Example to understand await page.route('**/api/profile', async route => { await route.fulfill({ status: 200, contentType: 'application/json', body: JSON.stringify({ name: 'Amina Khan', plan: 'Beginner' }) }); }); await page.goto('/profile'); await expect(page.getByText('Amina Khan')).toBeVisible();

Hour 2 — Hands-on task Mock a profile endpoint and test loading, empty and error states. Explain which tests should still call a real test API.

Answer / solution pattern await page.route('**/api/profile', route => route.fulfill({ status: 500, body: 'Server error' }) ); await page.goto('/profile'); await expect(page.getByRole('alert')).toContainText(/could not load|error/i);

Important questions

1. What can `page.route()` do?
2. What can a mock not prove?
3. Why test error states?

**Answers**

1. **What can `page.route()` do?**

   Intercept, modify, mock ( `fulfill` ), abort, or continue requests matching a URL pattern.

2. **What can a mock not prove?**

   That the real backend, database, or integration is correct—only that the UI handles the mocked response.

3. **Why test error states?**

   Users see errors too; UI must show alerts, retry, or empty states without breaking.

Revision notes • Mock only the dependency you intend to control. • Keep separate integration tests for real backend behavior. • Use patterns that match the intended URL precisely.

---

Day 21 — Isolation, parallel execution and reliable tests Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Playwright runs tests in workers. Parallel tests expose shared-state bugs, duplicate data, port conflicts and order dependencies. Use unique test data, isolated contexts and cleanup.

Example to understand module.exports = { workers: process.env.CI ? 2 : undefined, fullyParallel: true, retries: process.env.CI ? 1 : 0, use: { trace: 'retain-on-failure' } };

Hour 2 — Hands-on task Run your suite repeatedly and with multiple workers. Fix any test that depends on another test's data or execution order.

Answer / solution pattern npx playwright test --repeat-each=3 npx playwright test --workers=4 npx playwright test --workers=1

Important questions

1. Why can a suite pass with one worker and fail with four?
2. What is test isolation?
3. When are retries useful and dangerous?

**Answers**

1. **Why can a suite pass with one worker and fail with four?**

Shared accounts, duplicate unique keys, order-dependent data, or race on the same record when workers increase.

2. **What is test isolation?**

   Each test sets up its own data and does not rely on another test's side effects.

3. **When are retries useful and dangerous?**

   Useful in CI for transient network issues; dangerous if overused to hide locator/timing bugs without fixing root cause.

Revision notes • Retries are diagnostic resilience, not a substitute for fixing flakiness. • Create unique records and clean up after tests. • Avoid tests that depend on shared mutable accounts or ordering.

Weekly checkpoint — Week 3 Use a page object, fixtures, data-driven cases, one API test and one route-mocked UI test.

---

Day 22 — Test design: risk, coverage and maintainability Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Choose tests based on user impact and failure risk. A test plan should cover happy paths, negative paths, boundaries, permissions, persistence and important integrations. Avoid testing every pixel when behavior matters more.

Example to understand const scenarios = [ 'successful login', 'invalid password', 'required email', 'locked account', 'logout clears session' ]; // Turn each scenario into steps, data and an observable expected result.

Hour 2 — Hands-on task Draft a risk-based test plan for login and student creation. Rank scenarios high/medium/low and implement the three highest-risk cases.

Answer / solution pattern test('student creation persists in list', async ({ page }) => { await page.goto('/students'); await page.getByRole('button', { name: 'Add student' }).click(); await page.getByLabel('Name').fill('Test Student'); await page.getByLabel('Email').fill( `student-${Date.now()}@example.test` ); await page.getByRole('button', { name: 'Save' }).click(); await expect(page.getByRole('row', { name: /Test Student/ })).toBeVisible(); });

Important questions

1. How do you prioritize test cases?
2. What is the difference between smoke and regression testing?
3. What makes an expected result observable?

**Answers**

1. **How do you prioritize test cases?**

   By impact × likelihood: revenue paths, auth, data loss, and compliance first; cosmetic/low-risk later.

2. **What is the difference between smoke and regression testing?**

   Smoke = small fast gate on critical health; regression = broader suite covering many features and edge cases.

3. **What makes an expected result observable?**

Something a user or tester can see or query without reading server logs—message, URL, row, API field.

Revision notes • Automate stable, high-value workflows first. • A smoke suite should be fast and cover critical health. • Record assumptions and environment requirements.

---

Day 23 — Project A: login and account regression suite Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Project goal: create a small suite that covers valid login, invalid credentials, blank validation and logout. Keep test data configurable and assertions observable.

Example to understand test('invalid credentials are rejected', async ({ page }) => { await page.goto('/login'); await page.getByLabel('Email').fill('bad@example.test'); await page.getByLabel('Password').fill('wrong'); await page.getByRole('button', { name: 'Sign in' }).click(); await expect(page.getByRole('alert')).toBeVisible(); });

Hour 2 — Hands-on task Create 4-6 login-related tests, document test accounts and add a report. Do not automate a real service without permission.

Answer / solution pattern // Example logout assertion await page.getByRole('button', { name: 'Log out' }).click(); await expect(page).toHaveURL(/login/); await expect(page.getByRole('button', { name: 'Sign in' })).toBeVisible();

Important questions

1. Which login negative cases matter most?
2. How can tests avoid hard-coded credentials?
3. What artifacts should be included in the project README?

**Answers**

1. **Which login negative cases matter most?**

   Wrong password, unknown user, blank fields, locked account, expired session—anything that must not grant access.

2. **How can tests avoid hard-coded credentials?**

   `process.env.E2E_USER` / `E2E_PASSWORD` from `.env` (gitignored) or CI secrets—never literals in git.

3. **What artifacts should be included in the project README?**

   Setup, env vars, how to run tests/reports, test account policy, scope, and known limitations.

Revision notes • Deliverable: tests, test-case matrix, README and report. • Use environment variables for secrets. • A test must prove the resulting state, not only click buttons.

---

Day 24 — Project B: e-commerce cart and checkout Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Automate a realistic purchase flow in a safe demo environment: browse product, add to cart, verify quantity/total, submit test checkout and assert confirmation. Never submit real orders.

Example to understand await page.getByRole('button', { name: 'Add to cart' }).first().click(); await page.getByRole('link', { name: /cart/i }).click(); await expect(page.getByRole('heading', { name: /your cart/i })).toBeVisible(); await expect(page.getByText('1 item')).toBeVisible();

Hour 2 — Hands-on task Cover add item, remove item, empty cart, required checkout fields and successful demo checkout.

Answer / solution pattern await page.getByRole('button', { name: 'Checkout' }).click(); await page.getByLabel('First name').fill('Test'); await page.getByLabel('Last name').fill('Learner'); await page.getByLabel('Postal code').fill('500001'); await page.getByRole('button', { name: 'Continue' }).click(); await expect(page.getByRole('heading', { name: /overview/i })).toBeVisible();

Important questions

1. How would you verify a cart total?
2. Which checkout negative cases are important?
3. Why use a demo environment?

**Answers**

1. **How would you verify a cart total?**

   Parse displayed subtotal/tax/total text or `data-testid` totals; compare before/after quantity changes; do not trust a single static string if prices change.

2. **Which checkout negative cases are important?**

   Empty cart, invalid postal code, missing required fields, session timeout, payment failure simulation.

3. **Why use a demo environment?**

   No real money or inventory impact; predictable data; legal permission to automate.

Revision notes • Validate totals and quantities, not only page navigation. • Keep test orders non-production and clearly marked. • Test empty and boundary states.

---

Day 25 — Project C: student CRUD suite Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words CRUD means Create, Read, Update and Delete. A good test verifies each operation's result and persistence. API setup can create data quickly; UI assertions confirm user-facing behavior.

Example to understand test('search finds a student', async ({ page }) => { await page.goto('/students'); await page.getByRole('searchbox', { name: 'Search students' }).fill('Capstone Student'); await page.getByRole('button', { name: 'Search' }).click(); await expect(page.getByRole('row', { name: /Capstone Student/ })).toBeVisible(); });

Hour 2 — Hands-on task Implement create, list/search, update and delete tests using unique data. Confirm deletion in the UI and clean up safely.

Answer / solution pattern await page.getByRole('button', { name: 'Edit' }).click(); await page.getByLabel('Name').fill('Updated Capstone Student'); await page.getByRole('button', { name: 'Save changes' }).click(); await expect(page.getByRole('row', { name: /Updated Capstone Student/ })).toBeVisible();

Important questions

1. What is CRUD?
2. Why use unique record names?
3. How can you prove a deletion succeeded?

**Answers**

1. **What is CRUD?**

   Create, Read, Update, Delete—core persistence operations your admin/LMS features rely on.

2. **Why use unique record names?**

   Parallel tests and reruns won't clash on the same row name; search/delete targets stay unambiguous.

3. **How can you prove a deletion succeeded?**

   Assert row gone from UI, empty search results, or GET API returns 404/not in list after delete.

Revision notes • Deliverable: independent CRUD tests and cleanup strategy. • Verify persistence after create/update. • Do not rely on row indexes if a stable name is available.

---

Day 26 — Cross-browser testing: Chromium, Firefox, WebKit Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Playwright projects let the same suite run against multiple browser engines or device configurations. Differences can reveal compatibility bugs or brittle test assumptions.

Example to understand const { defineConfig, devices } = require('@playwright/test'); module.exports = defineConfig({ testDir: './tests', projects: [ { name: 'chromium', use: { ...devices['Desktop Chrome'] } }, { name: 'firefox', use: { ...devices['Desktop Firefox'] } }, { name: 'webkit', use: { ...devices['Desktop Safari'] } } ] });

Hour 2 — Hands-on task Run your highest-value login and CRUD tests in all three browser projects; inspect and document any browser-specific failure.

Answer / solution pattern npx playwright install npx playwright test --project=chromium npx playwright test --project=firefox npx playwright test --project=webkit

Important questions

1. What is a Playwright project?
2. Why might a test fail only in WebKit?
3. How should you investigate a browser-specific failure?

**Answers**

1. **What is a Playwright project?**

   A named browser/device configuration in `playwright.config` (e.g. chromium, firefox, webkit) with its own `use` options.

2. **Why might a test fail only in WebKit?**

   Different rendering, timing, or WebKit-specific behavior (date inputs, scroll, flex bugs) exposes app or test assumptions.

3. **How should you investigate a browser-specific failure?**

   Re-run with `--project=webkit`, open trace, compare snapshot and locator counts—fix app or test, avoid blind skip.

Revision notes • Install required browser binaries. • Do not hide genuine compatibility bugs with broad skips. • Reproduce and inspect trace on the failing project.

---

Day 27 — Responsive testing and accessibility basics Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Viewport settings simulate common screen sizes. Accessible roles and names help automation and people using assistive technology. Responsive emulation is useful but not a full substitute for real-device checks.

Example to understand test('navigation works on mobile viewport', async ({ page }) => { await page.setViewportSize({ width: 390, height: 844 }); await page.goto('/'); await expect(page.getByRole('heading', { level: 1 })).toBeVisible(); await expect(page.getByRole('button', { name: /menu/i })).toBeVisible(); });

Hour 2 — Hands-on task Test representative mobile, tablet and desktop viewports. Check that primary navigation and form controls remain visible and usable.

Answer / solution pattern await page.setViewportSize({ width: 768, height: 1024 }); await page.goto('/register'); await expect(page.getByLabel('Email')).toBeVisible(); await expect(page.getByRole('button', { name: 'Create account' })).toBeVisible();

Important questions

1. What is an accessible name?
2. Why test more than one viewport?
3. What does emulation not guarantee?

**Answers**

1. **What is an accessible name?**

   The name exposed to assistive tech—often from visible text, `aria-label`, or associated `<label>`.

2. **Why test more than one viewport?**

   Layout and navigation change (hamburger vs full nav); catch controls that overflow or hide on small screens.

3. **What does emulation not guarantee?**

   Real device GPUs, touch quirks, and OS fonts—emulation is representative, not exhaustive.

Revision notes • Prefer labels and semantic roles. • Test behavior and usability, not just pixel-perfect screenshots. • Choose representative breakpoints based on the app.

---

Day 28 — CI with GitHub Actions Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Continuous integration runs tests automatically when code changes. A typical pipeline installs Node dependencies and browser binaries, runs tests and uploads reports or traces as artifacts.

Example to understand

# .github/workflows/playwright.yml

name: Playwright tests on: [push, pull_request] jobs: test: runs-on: ubuntu-latest steps:

- uses: actions/checkout@v4
- uses: actions/setup-node@v4 with: node-version: 22 cache: npm
- run: npm ci
- run: npx playwright install --with-deps
- run: npx playwright test
- uses: actions/upload-artifact@v4 if: always() with: name: playwright-report path: playwright-report/

Hour 2 — Hands-on task Create a workflow that runs the suite on push and pull request. Verify reports upload even if tests fail.

Answer / solution pattern

# Local CI-equivalent checks

npm ci npx playwright install npx playwright test npx playwright show-report

Important questions

1. Why use `npm ci` in CI?
2. Why upload reports with `if: always()` ?
3. Where should secrets be stored?

**Answers**

1. **Why use `npm ci` in CI?**

   `npm ci` installs exactly from `package-lock.json` for reproducible CI builds (vs `npm install` which may resolve newer versions).

2. **Why upload reports with `if: always()` ?**

   You still want the HTML report when tests fail; `always()` uploads artifacts even on failure.

3. **Where should secrets be stored?**

   GitHub Actions secrets / vault env vars—never in repo, logs, or committed `.auth` files.

Revision notes • Pin versions thoughtfully and keep lockfiles committed. • Use repository secrets for credentials. • CI should use the same reliable test commands as local development.

Weekly checkpoint — Week 4 Run selected tests across browsers, test responsive layouts, and produce a CI workflow.

---

Day 29 — Final capstone planning and framework integration Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Combine page objects, fixtures, test data, API checks, reporting, trace-on-failure and cross-browser projects. A good framework is readable and maintainable rather than merely large.

Example to understand // Example structure // tests/ // auth.spec.js // students.spec.js // pages/ // login.page.js // students.page.js // fixtures/test.js // test-data/users.js // playwright.config.js // README.md

Hour 2 — Hands-on task Build a framework skeleton for an LMS/admin dashboard. Include login, student search and create/update flows, test data, README and a CI workflow.

Answer / solution pattern // Example reusable student search action async search(name) { await this.searchBox.fill(name); await this.searchButton.click(); }

Important questions

1. What belongs in a page object vs a test?
2. What must the README explain?
3. How do you keep test data safe and repeatable?

**Answers**

1. **What belongs in a page object vs a test?**

   Page object: locators + actions; test: scenario steps + assertions + why it matters.

2. **What must the README explain?**

   Prerequisites, URLs, env vars, install commands, test users policy, how to run smoke vs full, and CI behavior.

3. **How do you keep test data safe and repeatable?**

   Fixtures/JSON with fake data, unique IDs per run, gitignore secrets, teardown/delete after create tests.

Revision notes • Prioritize a small reliable suite over many flaky tests. • Document URLs, setup, test users, commands and known limitations. • Use trace and HTML report to diagnose failures.

---

Day 30 — Final capstone, review and job-readiness Learning outcome: By the end of this session, you should be able to explain and use the main idea in a working test.

Hour 1 — Key concept in simple words Your final goal is to explain, build, run and debug a suite. Include a risk-based test plan, UI tests, API tests, reusable page objects, cross-browser configuration, CI and a clear report.

Example to understand // Example combined UI + API verification const response = await request.get('/api/students'); expect(response.status()).toBe(200); const body = await response.json(); expect(body.data.some(student => student.name === 'Capstone Student')).toBeTruthy(); await page.goto('/students'); await page.getByRole('searchbox', { name: 'Search students' }).fill('Capstone Student'); await page.getByRole('button', { name: 'Search' }).click(); await expect(page.getByRole('row', { name: /Capstone Student/ })).toBeVisible();

Hour 2 — Hands-on task Publish a final project repository with 10+ meaningful tests, a test-case matrix, setup guide, HTML report evidence, screenshots/traces for a known failure, and a CI workflow.

Answer / solution pattern

# Final verification

npx playwright test npx playwright show-report

# Also run selected tests in Firefox and WebKit.

# Ensure .env, auth state, tokens and real user data are not committed.

Important questions

1. Explain one test from requirement through cleanup.
2. How do you debug a click timeout?
3. What does your suite cover that manual smoke testing might miss?

**Answers**

1. **Explain one test from requirement through cleanup.**

   Example: requirement ‘invalid login shows error’ → goto login → fill bad creds → click → expect alert → no dashboard URL → context closed automatically.

2. **How do you debug a click timeout?**

   Trace: locator count 0 vs many, overlay, wrong frame, not enabled—fix locator or wait for state, not blind timeout.

3. **What does your suite cover that manual smoke testing might miss?**

   Repeatable parallel regression, API contracts, precise edge cases, and evidence (report/trace) on every commit.

Revision notes • Job-ready means you can explain trade-offs, not memorize APIs. • Be prepared to live-code a locator, assertion and API check. • Never claim a mock proves the production backend works.

---

Real-world project workbook Build these projects in a demo app, a local app you own, or a test environment where you have permission. The answer snippets show the expected approach; selectors, routes and response structures must match the app.

Project 1 — Login and account regression suite Acceptance criteria • Valid login reaches the dashboard. • Invalid password shows a clear error. • Blank fields trigger validation. • Logout returns to login and blocks protected-page access. • Use environment variables for test credentials.

Solution pattern test('invalid login shows a user-facing error', async ({ page }) => { await page.goto('/login'); await page.getByLabel('Email').fill('wrong@example.test'); await

page.getByLabel('Password').fill('incorrect'); await page.getByRole('button', { name: 'Sign in' }).click(); await expect(page.getByRole('alert')).toBeVisible(); });

Answer checklist: use a dedicated test account; keep secrets in environment variables; cover positive and negative paths; assert resulting URL or visible message; document prerequisites.

Project 2 — E-commerce cart and checkout Acceptance criteria • Add an item and verify the cart count. • Remove an item and verify the empty state. • Check required checkout fields. • Complete a test checkout and verify confirmation. • Never place a real order.

Solution pattern await page.getByRole('button', { name: 'Add to cart' }).first().click(); await page.getByRole('link', { name: /cart/i }).click(); await expect(page.getByText('1 item')).toBeVisible(); await page.getByRole('button', { name: 'Checkout' }).click(); await expect(page.getByLabel('First name')).toBeVisible();

Answer checklist: assert count and totals; test empty and invalid states; use the app's demo payment path; ensure the test leaves no real financial side effects.

Project 3 — Student CRUD + search Acceptance criteria • Create a uniquely named student. • Search and verify the row. • Edit a field and verify the new value. • Delete the record and verify it is absent. • Clean up test data through approved API or UI.

---

Solution pattern const uniqueName = `PW Student ${Date.now()}`; await page.goto('/students'); await page.getByRole('button', { name: 'Add student' }).click(); await page.getByLabel('Name').fill(uniqueName); await page.getByLabel('Email').fill(`pw-${Date.now()}@example.test`); await page.getByRole('button', { name: 'Save' }).click(); await expect(page.getByRole('row', { name: new RegExp(uniqueName) })).toBeVisible();

Answer checklist: use unique data; verify persistence; don't depend on row positions; make tests independent; clean up safely.

Final Capstone — LMS/admin dashboard automation framework Acceptance criteria • Login suite and protected-route check. • Student list search and create/update workflow. • API test for list endpoint and response shape. • Page objects, fixtures and centralized config. • Chromium + Firefox + WebKit projects. • CI workflow, HTML report and failure trace. • README with setup, commands, assumptions and test data rules.

Solution pattern // UI and API evidence should agree: const response = await request.get('/api/students'); expect(response.status()).toBe(200); const body = await response.json(); expect(body.data.some(s => s.name === 'Capstone Student')).toBeTruthy(); await page.goto('/students'); await page.getByRole('searchbox', { name: 'Search students' }).fill('Capstone Student'); await page.getByRole('button', { name: 'Search' }).click(); await expect(page.getByRole('row', { name: /Capstone Student/ })).toBeVisible();

Capstone answer: submit a small reliable framework rather than a huge flaky suite. Include 10+ meaningful tests, a case matrix, trace/report evidence, safe test data and a CI pipeline.

---

Interview questions with answer guide

1. What is Playwright Test?

Answer: A test runner built for Playwright that provides test declarations, fixtures, assertions, hooks, parallel workers, retries and reporters. 2. Browser vs BrowserContext vs Page?

Answer: Browser is the browser engine instance; context is an isolated session; page is a tab/window within a context. 3. Why prefer getByRole and getByLabel?

Answer: They reflect how users and assistive technologies identify controls and are usually more resilient than styling-based selectors. 4. What is strict matching?

Answer: When a locator action expects one element but the locator matches multiple elements, Playwright reports ambiguity. Narrow the locator. 5. What is a web-first assertion?

Answer: An assertion that repeatedly checks a locator or page state until it passes or times out. 6. Why avoid fixed waits?

Answer: They can be too short on slow runs and waste time on fast runs. Wait for the expected state instead. 7. What makes a test independent?

Answer: It establishes its own preconditions, uses isolated/unique data, asserts its result and does not depend on another test's order. 8. beforeEach vs beforeAll?

Answer: beforeEach runs before each test; beforeAll runs once per worker scope. Shared mutable state in beforeAll can create coupling. 9. What belongs in a page object?

Answer: Page-specific locators and user-like actions. Tests should clearly state the scenario and its important assertions. 10. What is storage state?

Answer: A saved browser storage snapshot, commonly cookies and local storage, that can seed an authenticated context. 11. What is API testing with request?

Answer: Sending HTTP requests directly and asserting status, headers/body and application behavior without driving the UI. 12. What does page.route do?

Answer: Intercepts network requests to modify, mock or abort them. A mock does not prove the real backend is correct. 13. Why can parallel tests fail?

Answer: They may share accounts/data, collide on names, depend on order or compete for external resources. 14. How do you debug a click timeout?

Answer: Inspect the locator match count, visibility, enabled/stable state, overlays, URL and trace; reproduce the failure before changing waits. 15. What are traces useful for?

Answer: They capture actions, snapshots, timing and related evidence to reconstruct a failure.

---

16. Why test multiple browsers?

Answer: Different engines can expose compatibility issues and assumptions that only work in one browser. 17. What does accessibility-aware automation mean?

Answer: Using semantic roles and accessible names and checking that controls can be identified and used meaningfully. 18. What should CI do?

Answer: Check out code, install the intended Node version/dependencies/browsers, run tests, and preserve reports/artifacts. 19. What is a risk-based test plan?

Answer: A prioritized set of scenarios based on impact and likelihood, covering critical paths and high-risk failure modes. 20. What does a good capstone prove?

Answer: You can design, implement, run, debug and explain a maintainable suite and its limits.

---

Revision sheets Locator quick reference page.getByRole('button', { name: 'Save' }) page.getByRole('heading', { name: 'Dashboard' }) page.getByLabel('Email') page.getByPlaceholder('Search students') page.getByText('Saved successfully') page.getByTestId('save-button')

Actions and assertions await locator.click(); await locator.fill('text'); await locator.check(); await locator.uncheck(); await locator.selectOption({ label: 'Playwright' }); await expect(locator).toBeVisible(); await expect(locator).toHaveText('Saved'); await expect(page).toHaveURL(/dashboard/); await expect(locator).toBeEnabled();

Useful commands npx playwright test npx playwright test --headed npx playwright test --ui npx playwright test tests/login.spec.js npx playwright test --grep "valid user" npx playwright test --project=firefox npx playwright test --debug npx playwright show-report npx playwright show-trace trace.zip

Troubleshooting checklist • Locator matches zero elements: confirm the page, label, accessible name and whether the element appears after an action. • Strict-mode violation: locator matches more than one element; narrow it using a role/name, container or test ID. • Click timeout: inspect visibility, enabled state, overlays, animation, wrong frame/page and the trace. • Flaky assertion: replace arbitrary sleeps with a web-first assertion for the intended state. • Tests fail in parallel: remove shared mutable data, use unique records, isolate contexts and clean up. • Authentication expired: refresh storage state using a dedicated test account; never commit the state file. • API test fails: check endpoint, baseURL, status code, authorization and actual response JSON shape. • CI fails but local passes: compare Node/dependency versions, browser installation, environment variables, base URL and headless behavior. • Never fix every failure by increasing timeouts; diagnose the underlying cause.

---

30-day completion checklist ■ Initialized a JavaScript Playwright project and ran a first test. ■ Used variables, arrays, objects, loops and async/await in test code. ■ Explained Browser, BrowserContext and Page. ■ Used role, label, placeholder, text and test-ID locators appropriately. ■ Automated form controls and asserted outcomes. ■ Created test suites and configuration with baseURL. ■ Used state-based waits and removed routine fixed sleeps. ■ Handled dialogs, popups, uploads and downloads. ■ Inspected traces, screenshots and HTML reports. ■ Created a page object and custom fixture. ■ Used data-driven test cases and boundary values. ■ Used storage state safely for a dedicated test account. ■ Tested API status codes and response bodies. ■ Mocked a network response and documented mock limitations. ■ Ran tests repeatedly and in parallel. ■ Designed risk-based coverage and completed login, cart and CRUD projects. ■ Ran selected tests in Chromium, Firefox and WebKit. ■ Checked representative responsive viewports and semantic controls. ■ Created a CI workflow and captured reports. ■ Published a final repository with README, case matrix and 10+ meaningful tests.

Suggested final repository evidence README.md tests/

# specs

pages/

# page objects

fixtures/

# custom fixtures

test-data/

## non-secret data

playwright.config.js .github/workflows/playwright.yml playwright-report/

## generated; upload as CI artifact

test-cases.md

## scenario matrix

.gitignore

## .env, auth state, reports as appropriate

Official reference Use the official Playwright documentation for up-to-date syntax and supported configuration: https://playwright.dev/docs/intro. Also consult the test runner, locators, assertions, fixtures, API testing, authentication, trace viewer and CI guides.

End of guide — learn by typing, running, observing, debugging and explaining. A passing test is useful only when you understand what it proves.

---